ys7237 Yu-Fei Shih
tt3102 Ta-Chun Tai

Report

- Github link:

Part1:
In the first part of the assignment, I prepared and ran the starter Flask application and connected it to MongoDB as provided in the source code.

Part2:
In the second part, I containerized the application using Docker. The two-container design is:

- “web” container: Flask To-Do application
- “mongo” container: MongoDB database

I created the following files for this part:

- `Dockerfile`
- `docker-compose.yml`
- `.dockerignore`

The `Dockerfile` builds the Flask image by installing Python dependencies, copying the source code, and running `python app.py`.
The `docker-compose.yml` file defines the multi-container application, including the Flask service and the MongoDB service, container networking, environment variables, and persistent storage for MongoDB using a Docker volume.

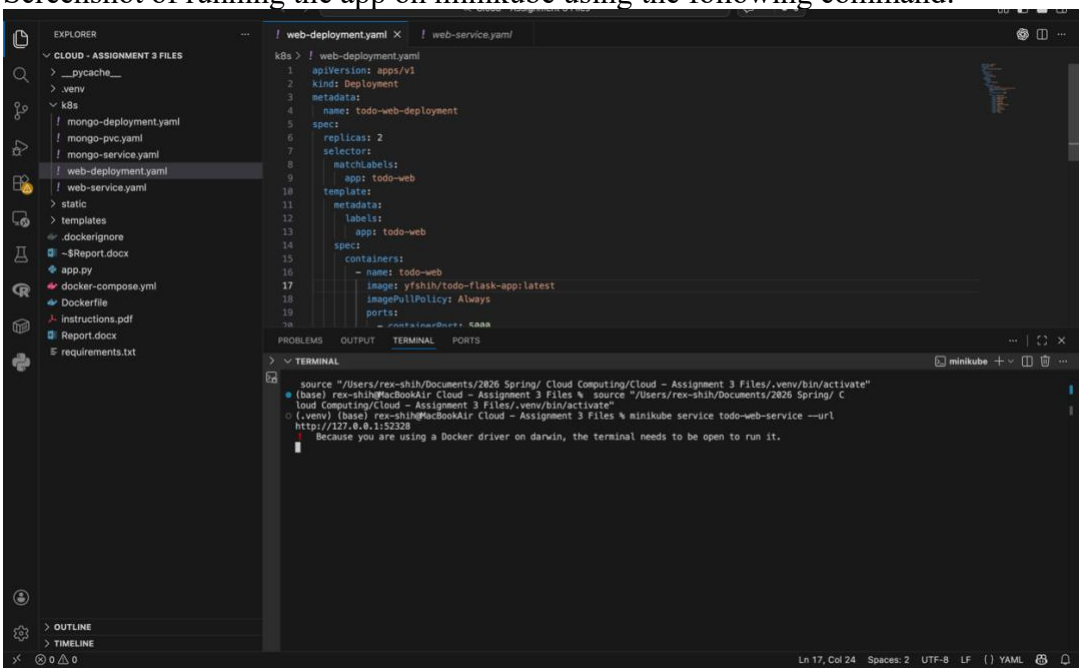
I then built and ran the containers using Docker Compose.

Part3
In the third part, I deployed the application on a local Kubernetes cluster using Minikube. I created Kubernetes manifests for both the Flask application and MongoDB. MongoDB was deployed with persistent storage, while the Flask application was deployed with multiple replicas.
I created the following Kubernetes resources:

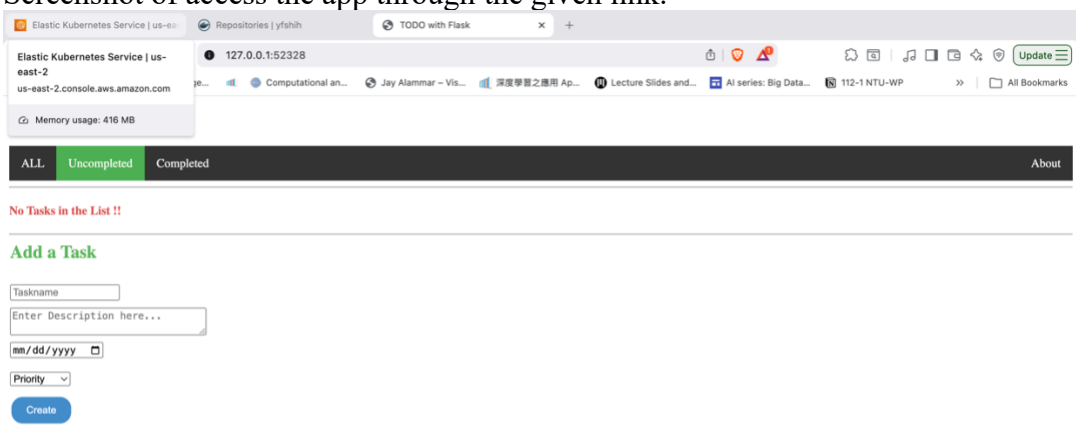
- MongoDB Deployment
- MongoDB Service
- MongoDB PersistentVolumeClaim
- Flask Deployment

ys7237 Yu-Fei Shih
tt3102 Ta-Chun Tai

1. Screenshot of running the app on minikube using the following command.



2. Screenshot of access the app through the given link.

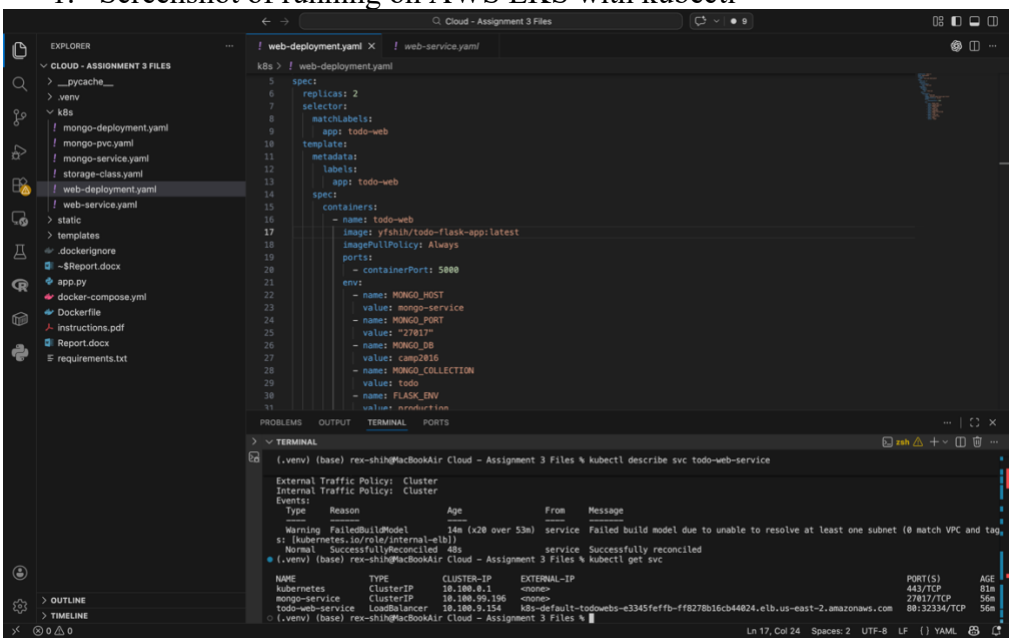


Part 4:
In the fourth part, I deployed the same application on Amazon EKS. I created an EKS cluster, configured “kubectl” to connect to the cluster, and reused the Kubernetes manifests with a few AWS-specific adjustments.

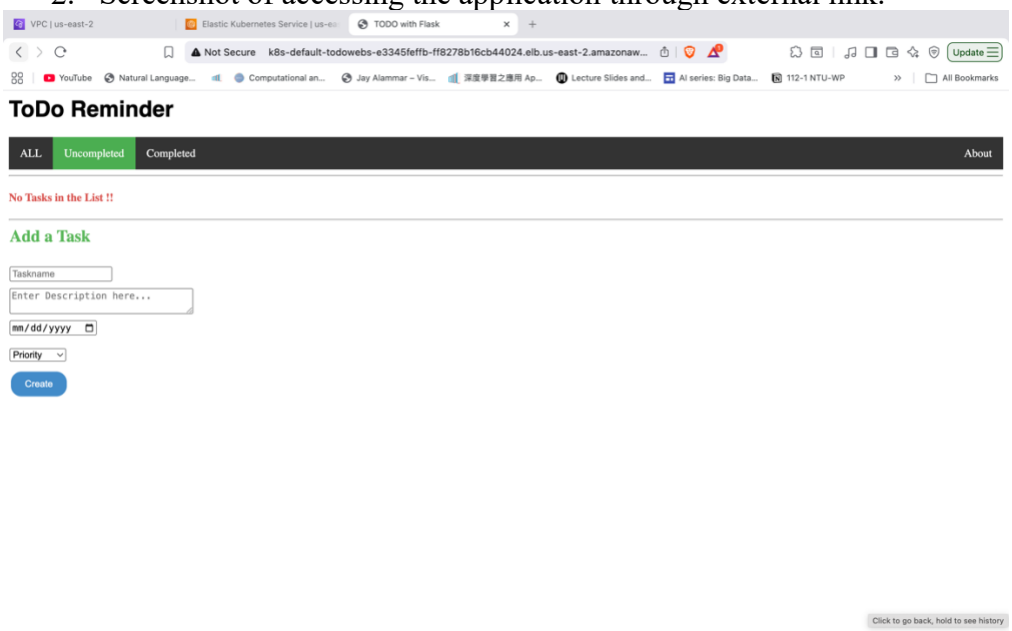
For AWS EKS, I set up the tag so that the EKS auto mode help me expose the application with external link. After deploying the application, AWS created an external load balancer endpoint that I used to open the application from the public internet.

ys7237 Yu-Fei Shih
tt3102 Ta-Chun Tai

1. Screenshot of running on AWS EKS with kubectl



2. Screenshot of accessing the application through external link.



Part 5:
In this part, my Flask deployment was configured with two replicas.

I then deleted one running Flask pod to test Kubernetes self-healing behavior. Kubernetes automatically created a replacement pod to maintain the desired replica count.

After that, I scaled the deployment up and down using the “kubectl scale deployment” command and verified that Kubernetes adjusted the number of running pods accordingly.

1. Screenshot of deleting a pod:

ys7237 Yu-Fei Shih
tt3102 Ta-Chun Tai

```
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get rs

NAME                              DESIRED   CURRENT   READY   AGE
mongo-deployment-5c6868989c       1         1         1       65m
todo-web-deployment-5f7c5cc89f    0         0         0       65m
todo-web-deployment-77fbdf6789    2         2         2       42m
todo-web-deployment-867689f4f8    0         0         0       47m
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
mongo-deployment-5c6868989c-scc94   1/1     Running   0          65m
todo-web-deployment-77fbdf6789-s82wp   1/1     Running   0          42m
todo-web-deployment-77fbdf6789-tq6p8   1/1     Running   0          37m
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl delete pod todo-web-deployment-77fbdf6789-s82wp
pod "todo-web-deployment-77fbdf6789-s82wp" deleted from default namespace
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
mongo-deployment-5c6868989c-scc94   1/1     Running   0          66m
todo-web-deployment-77fbdf6789-8r4dv   0/1     Pending   0          80s
todo-web-deployment-77fbdf6789-tq6p8   1/1     Running   0          39m
todo-web-deployment-77fbdf6789-8r4dv   0/1     Pending   0          2m5s
todo-web-deployment-77fbdf6789-8r4dv   0/1     ContainerCreating   0          2m6s
todo-web-deployment-77fbdf6789-8r4dv   1/1     Running             0          2m6s
```

2. Screenshot of scaling up:

```
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl scale deployment todo-web-deployment --replicas=3
deployment.apps/todo-web-deployment scaled
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mongo-deployment-5c6868989c-scc94   1/1     Running   0          76m
todo-web-deployment-77fbdf6789-8r4dv   1/1     Running   0          10m
todo-web-deployment-77fbdf6789-tq6p8   1/1     Running   0          48m
todo-web-deployment-77fbdf6789-vpw4v   0/1     Pending   0          11s
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mongo-deployment-5c6868989c-scc94   1/1     Running   0          77m
todo-web-deployment-77fbdf6789-8r4dv   1/1     Running   0          11m
todo-web-deployment-77fbdf6789-tq6p8   1/1     Running   0          50m
todo-web-deployment-77fbdf6789-vpw4v   0/1     Pending   0          93s
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mongo-deployment-5c6868989c-scc94   1/1     Running   0          82m
todo-web-deployment-77fbdf6789-8r4dv   1/1     Running   0          16m
todo-web-deployment-77fbdf6789-tq6p8   1/1     Running   0          54m
todo-web-deployment-77fbdf6789-vpw4v   1/1     Running   0          6m14s
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get rs

NAME                              DESIRED   CURRENT   READY   AGE
mongo-deployment-5c6868989c       1         1         1       82m
todo-web-deployment-5f7c5cc89f    0         0         0       82m
todo-web-deployment-77fbdf6789    3         3         3       59m
todo-web-deployment-867689f4f8    0         0         0       64m
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files %
```

3. Screenshot of scaling down:

```
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl scale deployment todo-web-deployment --replicas=2

deployment.apps/todo-web-deployment scaled
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get pods

NAME                                READY   STATUS    RESTARTS   AGE
mongo-deployment-5c6868989c-scc94   1/1     Running   0          85m
todo-web-deployment-77fbdf6789-8r4dv   1/1     Running   0          20m
todo-web-deployment-77fbdf6789-tq6p8   1/1     Running   0          58m
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get rs

NAME                              DESIRED   CURRENT   READY   AGE
mongo-deployment-5c6868989c       1         1         1       86m
todo-web-deployment-5f7c5cc89f    0         0         0       86m
todo-web-deployment-77fbdf6789    2         2         2       63m
todo-web-deployment-867689f4f8    0         0         0       68m
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files %
```

Part 6

In part 6, I configured the Flask deployment to use a rolling update strategy. I explicitly set:

- strategy.type: RollingUpdate
- maxUnavailable: 1
- maxSurge: 1

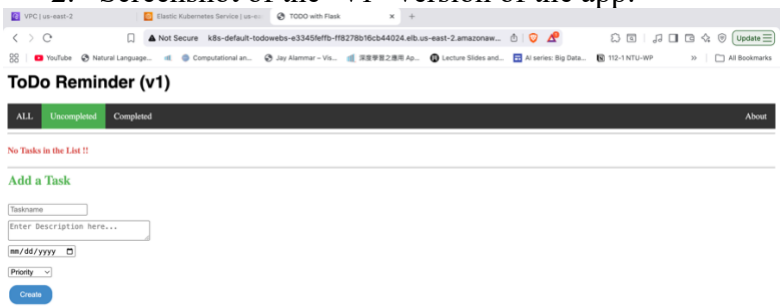
This configuration allows Kubernetes to update the application gradually while keeping the service available. I also versioned the application image and exposed the version in the user interface so I could visually verify the update.

ys7237 Yu-Fei Shih
tt3102 Ta-Chun Tai

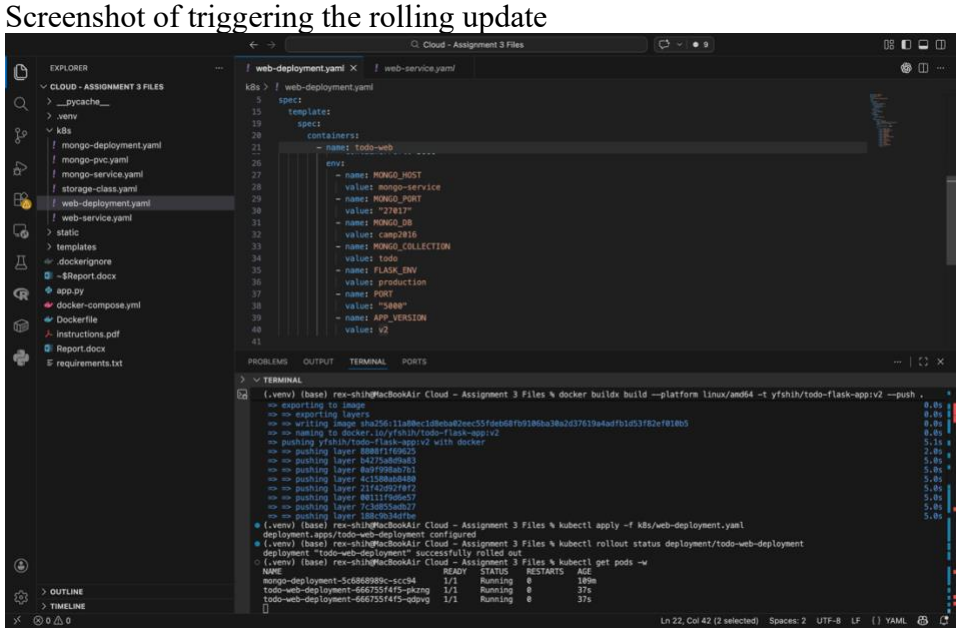
1. Screenshot of the part of the code that has rollingupdate added.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: todo-web-deployment
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app: todo-web
  template:
    metadata:
      labels:
```

2. Screenshot of the "v1" version of the app.

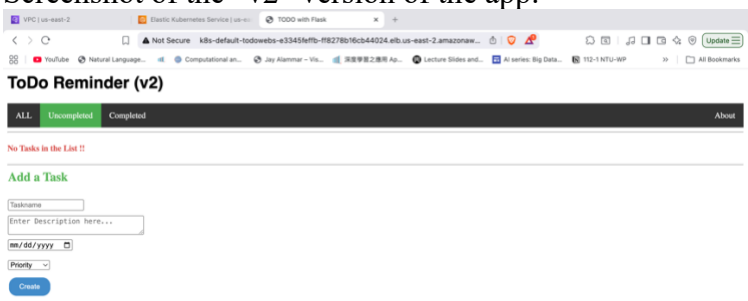


3. Screenshot of triggering the rolling update



ys7237 Yu-Fei Shih
tt3102 Ta-Chun Tai

4. Screenshot of the “v2” version of the app.



Part 7:
In part 7, I added health monitoring to the Flask application using Kubernetes liveness and readiness probes. I implemented two health endpoints in the Flask application:

- /healthz for liveness checking
- /readyz for readiness checking

The liveness probe checks whether the application container is alive. If the liveness probe fails, Kubernetes restarts the container. The readiness probe checks whether the application is ready to receive traffic. If the readiness probe fails, Kubernetes marks the pod as not ready and stops routing traffic to it. I also added environment variables to intentionally simulate liveness and readiness failures in order to verify probe behavior.

1. Screenshot of the new probes added to the code:

```
      - containerPort: 5000
    livenessProbe:
      httpGet:
        path: /healthz
        port: 5000
      initialDelaySeconds: 10
      periodSeconds: 10
      timeoutSeconds: 2
      failureThreshold: 3
    readinessProbe:
      httpGet:
        path: /readyz
        port: 5000
      initialDelaySeconds: 5
      periodSeconds: 5
      timeoutSeconds: 2
      failureThreshold: 3
    env:
      - name: MONGO_HOST
```

2. Screenshot of verifying the added probe

ys7237 Yu-Fei Shih
tt3102 Ta-Chun Tai

```
Annotations:      kubectl.kubernetes.io/restartedAt: 2026-03-23T02:41:10-04:00
Status:           Running
IP:               172.31.25.177
IPs:
  IP:           172.31.25.177
Controlled By:  ReplicaSet/todo-web-deployment-994f66bbf
Containers:
  todo-web:
    Container ID:   containerd://501400819cf865d93ee7597b2ed8603615e2148cf13a124e6f30d3f64ccdaa81
    Image:          yfshih/todo-flask-app:v3
    Image ID:       docker.io/yfshih/todo-flask-app@sha256:6ac49f2260d78f2c4da7e7c1dd9343980bd0234970a06ba4b7814fbd9e1c47b6
    Port:           5000/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Mon, 23 Mar 2026 04:17:59 -0400
    Ready:          True
    Restart Count:  0
    Liveness:       http-get http://:5000/healthz delay=10s timeout=2s period=10s #success=1 #failure=3
    Readiness:      http-get http://:5000/readyz delay=5s timeout=2s period=5s #success=1 #failure=3
    Environment:
      MONGO_HOST:        mongo-service
      MONGO_PORT:        27017
      MONGO_DB:          camp2016
      MONGO_COLLECTION:  todo
      FLASK_ENV:         production
      PORT:              5000
      APP_VERSION:       v3
      FAIL_LIVENESS:     false
      FAIL_READINESS:    false
```

3. Screenshot of triggering readiness failure.

```
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl set env deployment/todo-web-deployment FAIL_READINESS=true
deployment.apps/todo-web-deployment env updated
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get pods -w
NAME                                  READY   STATUS    RESTARTS   AGE
mongo-deployment-5c6868989c-scc94     1/1     Running   0          131m
todo-web-deployment-798bdb6d9b-ff66q  0/1     Running   0          33s
todo-web-deployment-798bdb6d9b-rlfpg  0/1     Running   0          33s
todo-web-deployment-994f66bbf-6wqrr   1/1     Running   0          12m
```

4. Screenshot of triggering liveness failure

```
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl set env deployment/todo-web-deployment FAIL_LIVENESS=true
deployment.apps/todo-web-deployment env updated
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get pods -w
NAME                                  READY   STATUS        RESTARTS   AGE
mongo-deployment-5c6868989c-scc94     1/1     Running       0          133m
todo-web-deployment-7c86598bb6-6tq2t  0/1     Running       0          6s
todo-web-deployment-7c86598bb6-lk6n8  0/1     Running       0          6s
todo-web-deployment-994f66bbf-6wqrr   1/1     Running       0          14m
todo-web-deployment-994f66bbf-tqkwb   1/1     Terminating   0          51s
todo-web-deployment-7c86598bb6-lk6n8  1/1     Running       0          9s
todo-web-deployment-7c86598bb6-6tq2t  1/1     Running       0          9s
todo-web-deployment-994f66bbf-6wqrr   1/1     Terminating   0          14m
todo-web-deployment-994f66bbf-6wqrr   1/1     Terminating   0          14m
todo-web-deployment-994f66bbf-tqkwb   0/1     Error         0          75s
todo-web-deployment-994f66bbf-tqkwb   0/1     Error         0          75s
todo-web-deployment-994f66bbf-tqkwb   0/1     Error         0          75s
todo-web-deployment-994f66bbf-6wqrr   0/1     Error         0          15m
todo-web-deployment-994f66bbf-6wqrr   0/1     Error         0          15m
todo-web-deployment-994f66bbf-6wqrr   0/1     Error         0          15m
todo-web-deployment-7c86598bb6-6tq2t  0/1     Running       1 (1s ago)   71s
todo-web-deployment-7c86598bb6-lk6n8  0/1     Running       1 (1s ago)   71s
todo-web-deployment-7c86598bb6-6tq2t  1/1     Running       1 (7s ago)   77s
todo-web-deployment-7c86598bb6-lk6n8  1/1     Running       1 (7s ago)   77s
todo-web-deployment-7c86598bb6-6tq2t  0/1     Running       2 (1s ago)   2m11s
todo-web-deployment-7c86598bb6-lk6n8  0/1     Running       2 (1s ago)   2m11s
todo-web-deployment-7c86598bb6-6tq2t  1/1     Running       2 (7s ago)   2m17s
todo-web-deployment-7c86598bb6-lk6n8  1/1     Running       2 (7s ago)   2m17s
```

Part 8:

For the extra credit part, I configured monitoring and alerting using Prometheus, Alertmanager, and Slack.

I installed kube-prometheus-stack in the monitoring namespace and configured Alertmanager to send notifications to Slack using an incoming webhook. Then I created a custom PrometheusRule to monitor the health of the Flask pods.

The custom alert rules included TodoWebPodNotReady and TodoWebPodRestarting. TodoWebPodRestarting rule detects whether the Flask pod restart count increases within a given time window.

ys7237 Yu-Fei Shih
tt3102 Ta-Chun Tai

1. Screenshot of the Pod-health.yaml

Cloud - Assignment 3 Files

web-deployment.yaml | alertmanager-values.yaml | pod-health-alerts.yaml

k8s > alerting > pod-health-alerts.yaml

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: pod-health-alerts
  namespace: monitoring
  labels:
    release: kube-prometheus-stack
spec:
  groups:
    - name: pod-health-alerts
      rules:
        - alert: TodoWebPodNotReady
          expr: kube_pod_status_ready{namespace="default", condition="true", pod=~"todo-web-deployment-.*"} == 0
          for: 2m
          labels:
            severity: warning
          annotations:
            summary: "ToDo web pod is not ready"
            description: "A todo-web pod has been unready for more than 2 minutes."
        - alert: TodoWebPodRestarting
          expr: increase(kube_pod_container_status_restarts_total{namespace="default", pod=~"todo-web-deployment-.*"}[5m]) > 0
          for: 1m
          labels:
            severity: critical
          annotations:
            summary: "ToDo web pod is restarting"
            description: "A todo-web pod restarted within the last 5 minutes."
```

2. Screenshot of Prometheus set up

```
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get pods -n monitoring
NAME                                                     READY   STATUS    RESTARTS   AGE
kube-prometheus-stack-grafana-7cb7cb8f78-74vvq           3/3     Running   0          104s
kube-prometheus-stack-kube-state-metrics-844c755cdf-qpqnf   1/1     Running   0          104s
kube-prometheus-stack-operator-59687599b-x4tgb           1/1     Running   0          104s
kube-prometheus-stack-prometheus-node-exporter-w48pf     1/1     Running   0          104s
kube-prometheus-stack-prometheus-node-exporter-wkkt8     1/1     Running   0          104s
kube-prometheus-stack-prometheus-node-exporter-zhv4h     1/1     Running   0          104s
prometheus-kube-prometheus-stack-prometheus-0            2/2     Running   0          96s
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files %
```

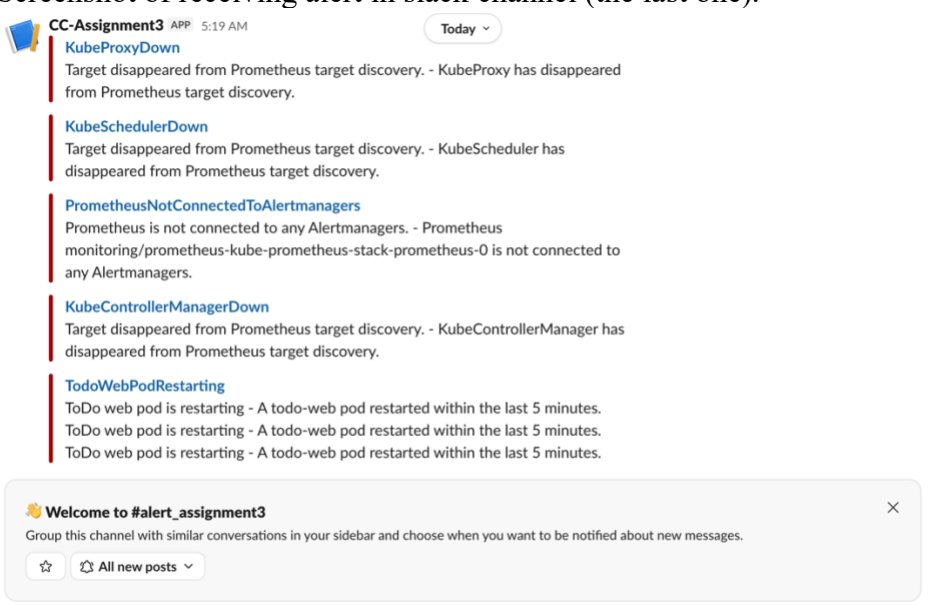
ys7237 Yu-Fei Shih
tt3102 Ta-Chun Tai

```
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get prometheusrule -n monitoring

NAME                                                                AGE
kube-prometheus-stack-alertmanager.rules                            3m56s
kube-prometheus-stack-config-reloaders                              3m56s
kube-prometheus-stack-etcd                                          3m56s
kube-prometheus-stack-general.rules                                 3m56s
kube-prometheus-stack-k8s.rules.container-cpu-usage-seconds-tot     3m56s
kube-prometheus-stack-k8s.rules.container-memory-cache              3m56s
kube-prometheus-stack-k8s.rules.container-memory-rss                3m55s
kube-prometheus-stack-k8s.rules.container-memory-swap               3m55s
kube-prometheus-stack-k8s.rules.container-memory-working-set-by     3m56s
kube-prometheus-stack-k8s.rules.container-resource                  3m56s
kube-prometheus-stack-k8s.rules.pod-owner                           3m56s
kube-prometheus-stack-kube-apiserver-availability.rules             3m56s
kube-prometheus-stack-kube-apiserver-burnrate.rules                 3m55s
kube-prometheus-stack-kube-apiserver-histogram.rules                3m55s
kube-prometheus-stack-kube-apiserver-slos                           3m55s
kube-prometheus-stack-kube-prometheus-general.rules                 3m56s
kube-prometheus-stack-kube-prometheus-node-recording.rules          3m55s
kube-prometheus-stack-kube-scheduler.rules                          3m55s
kube-prometheus-stack-kube-state-metrics                            3m55s
kube-prometheus-stack-kubelet.rules                                 3m55s
kube-prometheus-stack-kubernetes-apps                               3m55s
kube-prometheus-stack-kubernetes-resources                          3m55s
kube-prometheus-stack-kubernetes-storage                            3m55s
kube-prometheus-stack-kubernetes-system                             3m55s
kube-prometheus-stack-kubernetes-system-apiserver                   3m55s
kube-prometheus-stack-kubernetes-system-controller-manager          3m55s
kube-prometheus-stack-kubernetes-system-kube-proxy                  3m55s
kube-prometheus-stack-kubernetes-system-kubelet                     3m55s
kube-prometheus-stack-kubernetes-system-scheduler                   3m55s
kube-prometheus-stack-node-exporter                                 3m55s
kube-prometheus-stack-node-exporter.rules                           3m55s
kube-prometheus-stack-node-network                                  3m55s
kube-prometheus-stack-node.rules                                    3m56s
kube-prometheus-stack-prometheus                                    3m55s
kube-prometheus-stack-prometheus-operator                           3m55s
pod-health-alerts                                                   2m28s
```

3. Screenshot for triggering alert.

```
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl set env deployment/todo-web-deployment FAIL_LIVENESS=true

(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files % kubectl get pod
NAME                                  READY   STATUS    RESTARTS      AGE
mongo-deployment-5c6868989c-scc94     1/1     Running   0             157m
todo-web-deployment-7c86598bb6-5hqp9  1/1     Running   2 (48s ago)   3m8s
todo-web-deployment-7c86598bb6-m2mm8  1/1     Running   2 (48s ago)   3m8s
(.venv) (base) rex-shih@MacBookAir Cloud - Assignment 3 Files %
```

4. Screenshot of receiving alert in slack channel (the last one).